Ders 13 Genişletilmiş Notu: Öğrenci Proje Sunumları: Hash Fonksiyonlarına Saldırılar, Evrensel Hash, Algısal Hash ve Blokzincir

Ders 13 Genişletilmiş Notu: Öğrenci Proje Sunumları: Hash Fonksiyonlarına Saldırılar, Evrensel Hash, Algısal Hash ve Blokzincir

> Bu not, ders kaydının altyazısından hazırlanmış ayrıntılı ders notudur. Kısa tekrar için aynı dersin kısa notuna bakın.

Bu ders, bir önceki dersteki gibi öğrenci proje sunumlarına ayrılmıştır. On iki sunum yapıldı. Konular dört başlıkta toplanıyordu: hash fonksiyonlarının saldırılara karşı dayanıklılığı, çakışmaya dayanıklı hash fonksiyonları ve evrensel hash, algısal hash (perceptual hashing) ve blokzincirde hash fonksiyonları. Not yalnız sunumların akademik içeriğini ve hocanın akademik yorumlarını aktarır. Sunum yapanlar "bir öğrenci" ya da "bir grup" diye anılır. Aynı konu birkaç kez anlatıldığı için tekrar eden tanımlar ilk geçtikleri yerde ayrıntılı, sonra kısa verilmiştir.

İçindekiler

- 1. Hash Fonksiyonlarının Saldırı Dirençlerine Göre Karşılaştırılması (Birinci Sunum)
- 2. Çakışmaya Dayanıklı Hash Fonksiyonları: Giriş (İkinci Sunum)
- 3. Algısal Hash: Literatür ve Nesne Takibi (Üçüncü Sunum)
- 4. Blokzincirde Hash Fonksiyonları ve Proof of Work (Dördüncü Sunum)
- 5. Evrensel Hash Fonksiyonları (Beşinci Sunum)
- 6. Çakışmaya Dayanıklı Hash Fonksiyonları: Tasarım ve Tarihçe (Altıncı Sunum)
- 7. Hash Fonksiyonlarının Ataklara Dayanıklılığı (Yedinci Sunum)
- 8. Algısal Özetleme: aHash, dHash, pHash, wHash (Sekizinci Sunum)
- 9. Blokzincir Mimarisi ve Kullanılan Hash Fonksiyonları (Dokuzuncu Sunum)
- 10. Algısal Hash: Aşamalar ve Uygulama Alanları (Onuncu Sunum)
- 11. Hash Algoritmalarının Güçlü ve Zayıf Yönleri (On Birinci Sunum)
- 12. SHA-256'nın Adımları ve Blokzincir (On İkinci Sunum)
- Örnek Sorular

1. Hash Fonksiyonlarının Saldırı Dirençlerine Göre Karşılaştırılması (Birinci Sunum)

Sunum dört bölümden oluşuyordu: hash fonksiyonlarına genel giriş, bir hash fonksiyonunun güvenli sayılması için gereken ölçütler, standart hash fonksiyonları ve saldırı dirençleri, son olarak standart dışı yeni yöntemler.

1.1 Hash Fonksiyonu ve Güvenlik Ölçütleri

Hash fonksiyonları kredi kartı işlemlerinden indirilen dosyanın bütünlüğünü kontrol etmeye (checksum) kadar dijital ortamda pek çok yerde kullanılan matematiksel fonksiyonlardır. Fonksiyon olarak bakıldığında herhangi boyuttaki bir girdiyi sabit boyutta bir çıktıya çevirirler. Ama bu şartı sağlayan her fonksiyon güvenli bir hash fonksiyonu değildir; belirli güvenlik ölçütlerini

sağlaması gerekir. Öğrenci bu ölçütleri, hash fonksiyonlarının maruz kalabileceği saldırılar üzerinden anlattı ve hepsini aynı basit örnekle gösterdi: $H(x) = x \bmod 3$.

- **Çakışma (collision) saldırısı:** İki farklı girdinin aynı hash değerini vermesidir. $H(5) = 5 \bmod 3 = 2$ ve $H(8) = 8 \bmod 3 = 2$ olduğu için 5 ve 8 çakışır. Böyle bir ikilinin (ya da daha fazla girdinin) kolayca bulunabilmesi, fonksiyonun çakışma direncinin düşük olduğunu gösterir.
- **Ön görüntü (preimage) saldırısı:** Bir çıktı (hash değeri) verilir ve bu çıktıyı veren girdi aranır. Çıktı 2 verilmişse 5 bir ön görüntüdür; bu örnekte geriye dönük işlem çok kolaydır.
- **İkinci ön görüntü (second preimage) saldırısı:** Bu kez bir girdi verilir ve aynı çıktıyı veren başka bir girdi aranır. 8 verilmişse $H(8) = 2$'dir; 2'yi veren başka bir girdi olarak 5 bulunur. 5, 8'in ikinci ön görüntüsüdür.

Bir hash fonksiyonu bu üç saldırıya dirençli olmalı, aynı zamanda makul kaynak kullanmalıdır. Bir hash değerini üretmek çok uzun sürüyorsa fonksiyon pratikte kullanılamaz.

1.2 Hash Fonksiyonlarının Özellikleri ve Merkle-Damgård Yapısı

Öğrenci algoritmaları karşılaştırmak için dört özellik kullandı:

- **Blok boyutu:** Fonksiyonun bir seferde işlediği girdi parçasının boyutu.
- **Döngü (tur) sayısı:** Girdiye uygulanan işlemler yinelemeli (iteratif) olarak tekrarlanır; bir turun çıktısı bir sonraki tura girer.
- **Çıktı boyutu:** En önemli özelliklerden biridir, çünkü saldırganın eline geçen bilgi bu çıktıdır.
- **Yapı:** Çoğu algoritma Merkle-Damgård yapısını kullanır. Bu yapıda kısa kalan mesaj dolgu (padding) ile belirli bir sabit boyuta tamamlanır, ardından üzerine yinelemeli olarak bir dizi işlem uygulanır.

1.3 Standart Algoritmalar

MD5 (Message Digest 5). Adından anlaşılacağı gibi bir mesajın özetini çıkarmayı amaçlar; MD1'den başlayan bir ailenin son sürümüdür. Slayttaki tablo (ekrandan, 00:10:00): blok boyutu 512 bit, çıktı boyutu 128 bit, döngü sayısı 18, yapı Merkle-Damgård. Maruz kaldığı saldırılar: çakışma ve hızlı çakışma. Öğrenci hızlı çakışmada gereken karmaşıklığın normal çakışmaya göre çok düştüğünü söyledi; bu yüzden MD5 günümüzde güvenli sayılmaz.

> **Ders sonrası not:** Slayttaki "döngü sayısı 18" değeri yanlıştır. MD5'te 4 tur ve her turda 16 adım, toplam 64 işlem vardır; on birinci sunumdaki karşılaştırma tablosu da 64 yazıyor (ekrandan, 02:40:00).

RIPEMD-160. İki paralel MD4 benzeri hattın çalıştırılıp sonuçlarının birleştirilmesiyle geliştirilmiştir. Blok boyutu 512 bit, çıktı 160 bit, yapı Merkle-Damgård. Zayıf olduğu saldırılar: çakışma ve ön görüntü.

> **Ders sonrası not:** Sunumda çıktının MD5'e göre "128 bit daha fazla" olduğu söylendi. Fark $160 - 128 = 32$ bittir. Tur sayısı RIPEMD-160'ta büyük olasılıkla 80'dir (5 tur × 16 adım).

SHA-1 (Secure Hash Algorithm 1). SHA ailesinin ilk yaygın üyesidir. Blok boyutu 512 bit, çıktı 160 bit, döngü sayısı 80, yapı Merkle-Damgård; özellik olarak RIPEMD-160'a benzer. Maruz kaldığı saldırılar çakışma ve freestart çakışmadır. Merkle-Damgård algoritmalarının başlamak için bir başlangıç vektörüne (IV) ihtiyacı vardır. Saldırganın bu başlangıç vektörünü kendisi belirleyerek algoritmayı zayıflatmasına freestart çakışma denir.

SHA-2. SHA-1'in üzerine geliştirilmiştir. Sunumda 256 ve 512 sürümleri anlatıldı; sürüm adı çıktı boyutunu gösterir. SHA-512'nin blok boyutu 1024 bit ve tur sayısı 80, SHA-256'nın tur sayısı 64'tür. Daha karmaşık olması için doğrusal olmayan fonksiyonlar eklenmiştir; bunun bedeli hesaplama performansının düşmesidir. Sunumda kripto para alanında kullanıldığı söylendi. Bilinen zayıflığı ön görüntü saldırıları olarak verildi.

SHA-3. Düzenlenen bir yarışmada geliştirilmiş, önceki algoritmalardan çok farklı çalışan bir algoritmadır. Sunumda 256 ve 512 sürümleri anlatıldı: SHA3-256'nın blok (girdi) boyutu 1088 bit, SHA3-512'ninki 576 bittir; tur sayısı 24'tür. Tur sayısı düşük görünse de yapı farklıdır:

önceki algoritmalar Merkle-Damgård kullanırken SHA-3 sünger (sponge) yapısını kullanır. Bu yapı çok sayıda rastgele permütasyon üretebilen esnek bir yapıdır ve istenen boyutta girdi alıp istenen boyutta çıktı üretebilir. Günümüzde önerilen standart olarak görülür. Maruz kaldığı saldırılar: SHA3-256 için pratik çakışma (makul kaynaklarla bulunabilen çakışma) ve yakın çakışma (near-collision: iki farklı girdinin aynı değil ama çok benzer çıktı vermesi). SHA3-512 için yalnız olası bir ilk çakışmadan söz edildi. Öğrenci SHA-3'ü bugün için en güvenli algoritma olarak niteledi.

> **Ders sonrası not:** Sunumda SHA-3'ün iki sürümünün de "çıktısının 1600 bit" olduğu söylendi. 1600 bit Keccak'ın iç durum (state) boyutudur; çıktı SHA3-256'da 256, SHA3-512'de 512 bittir (ekrandan, 01:45:40).

1.4 Standart Dışı Yeni Yöntemler

Öğrenci, standart hash fonksiyonları dışında önerilen dört çalışmayı anlattı:

- **Hücresel otomat tabanlı hash:** Merkle-Damgård temelli; basit kurallardan karmaşık davranış üretebilen hücresel otomatların bu yeteneği hash fonksiyonuna uygulanmış. İstatistiksel olarak iyi sonuç vermiş ama pratikte yeterince test edilmemiş.
- **Değişken boyutlu hash:** Kullanıcı istediği güvenlik seviyesine göre çıktı boyutunu (örneğin 256 ya da 1024 bit) ve girdinin geçeceği döngü sayısını seçebiliyor. MD5 tabanlı olduğu için MD5'in sorunlarını taşıyor.
- **Kaotik harita tabanlı hash:** Kaotik haritalar bir çekirdek değerden çok çeşitli çıktılar üretebilen fonksiyonlardır; bu, hash fonksiyonlarının ihtiyaç duyduğu bir özelliktir. Ama geliştirilen algoritma çakışma saldırılarına karşı zayıf çıkmış.
- **Sinir ağı tabanlı hash:** Yapay sinir ağının eğitimde oluşan ağırlıkları dışarıdan kolayca bulunamaz; bu ağırlıklar gizli bir anahtar gibi kullanılarak 128 bitlik çıktı üreten bir hash geliştirilmiş. Sinir ağı işin içine girince hesaplama zor ve uzun sürüyor; 128 bit de günümüzde güvenlik açısından kısa kalıyor.

Sunumun genel sonucu: hem standart hem yeni yöntemlerin farklı zayıflıkları vardır. Yeni yöntemlerin en büyük zayıflığı yeterince test edilmemiş olmalarıdır. Eski yöntemler uzun süredir herkese açık olduğu için pek çok saldırıya maruz kalmış ve kendini ispatlamıştır; yeni yöntemlerin henüz böyle bir geçmişi yoktur.

1.5 Hocanın Yorumu

Hoca özellikle güncel makalelerden söz edilen kısım için teşekkür etti; sunumlarda güncel çalışmalara değinilmesini istediğini hatırlattı. Sunum açısından bir geri bildirim verdi: çalışmalar anlatılırken çalışma adları slaytın üstüne yazılsaydı dinleyenler adlara bakıp konuya daha rahat girebilirdi.

2. Çakışmaya Dayanıklı Hash Fonksiyonları: Giriş (İkinci Sunum)

İkinci sunumun konusu çakışmaya dayanıklı hash fonksiyonlarıydı (collision resistant hash functions).

2.1 Tanım ve Kullanım

Hash fonksiyonları, girdinin boyutundan bağımsız olarak belirli uzunlukta çıktı üreten tek yönlü algoritmalardır. Çıktı girdiye bağlıdır ve ona özeldir; aynı fonksiyona aynı girdi her verildiğinde aynı çıktı oluşur. Çıktıya message digest, hash, checksum ya da dijital parmak izi (digital fingerprint) denir. Kullanım alanları: veri bütünlüğünü doğrulama, parola saklama, sayısal imza, mesaj doğrulama.

En bilinen kullanımı web uygulamalarında parola saklamadır. Kayıt sırasında kullanıcının parolasının hash'i alınır ve kullanıcı adıyla birlikte veritabanında saklanır. Girişte verilen parolanın hash'i yeniden alınır ve kayıtlı hash'le karşılaştırılır; eşitse giriş izni verilir. Veritabanına bir saldırı olsa bile parolanın kendisi değil hash'i ele geçer.

Ders sonrası not: Sunumda “iki farklı girdinin çıktıları aynı olamaz” dendi. Girdi uzayı çıktı uzayından büyük olduğu için çakışmalar her zaman vardır; istenen, bunların bulunmasının hesaplama açısından zor olmasıdır. Altıncı ve yedinci sunumlar bunu açıkça belirtti.

2.2 Hash Aileleri ve Bilinen Kırılmalar

Sunumda sayılan hash aileleri: MD (Message Digest) ailesi, HAVAL, SHA (Secure Hash) ailesi, Whirlpool ve RIPEMD.

- **MD5:** MIT’den Ron Rivest tarafından geliştirildi, 128 bit çıktı verir. 2004’te başlatılan bir çalışmayla çakışmalara karşı güvenli olmadığı ortaya çıktı.
- **HAVAL:** 15 farklı güvenlik seviyesini destekleyen tek yönlü bir hash algoritmasıdır; 1992’de yayımlandı.
- **SHA-1:** ABD Ulusal Güvenlik Ajansı (NSA) tarafından geliştirildi, NIST desteğiyle ABD’de standart oldu; 160 bit çıktı verir. 2017’de yaklaşık 6610 yıllık işlemci zamanı kullanılarak bir çakışma bulundu.

  Ders sonrası not: 2017’deki SHA-1 çakışması (SHAttered) için bildirilen hesaplama büyük olasılıkla yaklaşık 6500 CPU yılı ve 110 GPU yılıdır; sunumdaki 6610 yıl bu ikisinin toplamına karşılık gelir.

Bu çakışmalar yüzünden MD5 ve SHA-1 günümüzde güvenli kabul edilmez. Sunumdaki ilke: bir hash fonksiyonu, herhangi bir çakışma bulunana kadar çakışmaya karşı korunaklı kabul edilir; çakışma bulunduğunda güvensiz hale gelir.

2.3 Üç Özellik ve Çığ Etkisi Örnekleri

Güvenli kabul edilmek için gereken üç özellik birinci sunumdakiyle aynıdır: çakışma direnci, ön görüntü direnci (hash değerinden orijinal girdinin elde edilememesi) ve ikinci ön görüntü direnci. İkinci ön görüntünün çakışmadan farkı şöyle anlatıldı: aynı hash’i veren iki girdiyi rastgele aramak yerine, verilmiş bir girdinin hash’ini veren başka bir girdi aranır.

Ardından görsel örnekler gösterildi. Bir kedi görüntüsünün hash’i alındı; sonra görüntüden bıyıklarından biri silindi. Görüntüdeki fark çok küçük olmasına rağmen oluşan hash tamamen farklıydı. Öğrenci, görüntü işlemeyle uğraşan biri olarak bunun dikkatini çektiğini söyledi: görüntüden özellik çıkarılarak hash üretildiği düşünülse hash’in bir kısmının aynı kalması beklenebilirdi; ama kriptografik hash’te en küçük fark bile tamamen farklı bir hash üretmelidir.

2.4 Kaba Kuvvetle Çakışma Aramanın Maliyeti

Slaytta “Hello1!”, “Hello2!”, “Hello3!”, …, “Hello9998!”, “Hello9999!”, “Hello10000!”, bir kedi resmi ve “My entire music collection” girdilerinin hash’leri listelenmiş; hiçbiri hedef hash’i vermemişti (ekrandan, 00:28:45). Slayttaki hesap: normal bir bilgisayar saniyede yaklaşık 60 milyon hash hesaplayabilir; bir çözüm bulmak için beklenen deneme sayısı 2^{255}’tir. Süre $2^{255}/(60 \cdot 10^6)$ saniye, yani yaklaşık $3 \cdot 10^{61}$ yıldır. Bu, tek bir bilgisayarla imkansızdır. Hesap Örnek Sorular bölümünde adım adım verilmiştir.

2.5 E-postada Görüntü Bütünlüğü

Son örnek bir e-posta senaryosuydu. Gönderilecek görüntünün hash’i gönderimden önce alınır ve görüntüyle birlikte e-postaya eklenir. Alıcı tarafta görüntünün hash’i aynı algoritmayla yeniden alınır ve iki hash karşılaştırılır. Eşitse görüntü bozulmadan ve bir saldırıyla değiştirilmeden ulaşmıştır.

3. Algısal Hash: Literatür ve Nesne Takibi (Üçüncü Sunum)

Üçüncü sunumu iki kişilik bir grup yaptı. Konu algısal hash (perceptual hashing) üzerine okunan makalelerdi.

3.1 Algısal Hash Nedir?

Algısal hash, görsellerden hash üretmek için kullanılan bir yöntemdir. Önemli olan görüntünün içeriğidir: içerik biraz değiştiğinde hash'in de yalnız biraz değişmesi istenir. Grup bir algısal hash algoritmasından beklenen beş özelliği sıraladı:

- **Ayırt edicilik (distinct):** Biraz benzer de olsa farklı iki görüntünün hash'i farklı olmalıdır.
- **Dayanıklılık (resilient):** Değişim az olduğunda iki hash arasındaki fark da az olmalıdır.
- **Deterministik olma:** Aynı fotoğraf iki kez hash'lenince aynı sonuç çıkmalıdır.
- **Verimlilik (efficient).**
- **Geri döndürülemezlik (non-reversible):** Zararlı içeriğin hash'i, benzerlerini engellemek için sosyal medya platformlarına dağıtılabilir. Hash ele geçse bile zararlı içeriğin kendisine ulaşılamamalıdır.

Soru (hoca): Dayanıklılık özelliği hangi uygulamalar için önemlidir? Yani hash'in iki farklı resim için farklı, ama biraz benzerlik de içeren bir değer üretmesi nerede gerekir?

Cevap: Nesne takibi (object tracking). Nesne sağa sola hareket ederken aslında yalnız konumu değişir; ardışık karelerdeki hash'lerin birbirine yakın olması beklenir. Ayrıca bir görüntü bulanıklaştırıldığında ya da biraz değiştirildiğinde içerik hâlâ aynıdır; dayanıklılık bunu yakalar. Hoca, bu özelliğin klasik hash'ten farklı olduğu için açıklanmasının iyi olacağını söyledi.

3.2 Birinci ve İkinci Makale: İçerik Moderasyonu

Birinci makale, internette denetimden (moderasyon) geçmeyen içeriklerin yayılması sorununu ele alıyordu. Her gün yüklenen içerik insanlara denetletilemeyecek kadar fazladır; bir yapay zeka modeliyle yapmak da yüksek maliyetlidir. Bu yüzden algısal hash kullanılmıştır.

Makalede beş özellik üzerinden klasik ("hard") hash ile algısal hash karşılaştırılır. MD5 ve SHA-1 gibi hash'ler resimdeki en küçük değişiklikte rastgele başka bir değer üretir; içeriğin varyasyonları tamamen farklı hash'ler verir, yani dayanıklı değildirler. Algısal hash benzer görüntülerden benzer hash üretir ve beş özelliği de sağlar. Grup bunu insanın görme sistemini biraz taklit etmek diye özetledi: içeriğe göre bakar. Slayttaki örnekte üzerinde "0" yazan bir görsel giderek bulanıklaştırılmıştı. Orijinalle kendisinin uzaklığı 0, biraz bulanıklaştırılmış hâliyle uzaklığı 0,08'di; bulanıklık arttıkça uzaklık da arttı.

Kullanım alanları: sosyal medyadaki içeriklerin denetlenmesi ve zararlı olanların filtrelenmesi, birbirine çok benzeyen resimlerin tespiti, biraz değiştirilmiş telifli içeriğin tespiti (klasik hash'ten kaçar ama algısal hash'ten kaçmaz), adli analizde delillerin karşılaştırılması. Araştırmacılar yöntem yapay zeka modelleriyle birleştikçe ve doğruluğu arttıkça hayatımıza daha çok gireceğini öngörüyor.

İkinci makale, sosyal ağlarda zararlı görüntülerin otomatik filtrelenmesi için daha ayrıntılı bir iş hattı (pipeline) öneriyordu; derin öğrenme ve algısal eşleştirme birlikte kullanılıyordu:

1. Sosyal medyadan toplanan gönderilerden bir veri seti oluşturulur.
2. Algısal hash ile yinelenen görüntüler ayıklanır: aynı resmin bulanıklaştırılmış, üzerine yazı eklenmiş, kırpılmış ya da yeniden boyutlandırılmış hâlleri veri setinden çıkarılır.
3. Azalan veri insanlar tarafından etiketlenir.
4. Bu etiketlerle bir makine öğrenmesi modeli eğitilir; model artık insanın yaptığı işi yapabilir hale gelir.

Örnek uygulama, depremde oluşan hasarın boyutunun modele öğretilmesiydi. Böylece insanların uzmanlığı verimli kullanılır ve insanlara düşen yük azalır.

3.3 Üçüncü Makale: Ön Plan Tespiti ve Algısal Hash ile Görsel Takip

Grubun ikinci üyesi algısal hash'in ön plan tespiti ve nesne takibi gibi özelliklerini kullanan iki makaleyi anlattı. İlki "Visual tracking based on improved foreground detection and perceptual hashing" başlıklı, Neurocomputing dergisinde 2015'te yayımlanmış çalışmaydı (ekrandan, 00:40:00). Slayttaki maddeler: dijital görüntü işleme için algısal hash, geliştirilmiş üç kare farkı (improved three-frame differencing), yavaş ve hızlı hareket eden nesnelerin tespiti, alternatiflerine göre daha basit ve maliyet etkin olma.

Makalede kullanılan üç algısal hash yöntemi:

- **aHash (average hash):** Görsel gri seviyeye çevrilir, 8×8'e küçültülür, piksellerin ortalaması alınır. Her piksel ortalamayla karşılaştırılır; büyükse 1, küçükse 0 yazılır. Yalnız ortalamayla kıyas vardır, piksellerin kendi arasındaki ilişkiye bakılmaz.
- **pHash:** aHash'e göre daha dayanıklıdır. Ortalamadan önce ayrık kosinüs dönüşümü (DCT) uygulanır; DCT bir frekans ölçüsüdür ve komşu piksellerin ilişkisini hesaba katar.
- **dHash:** pHash'e benzer biçimde piksellerin birbiriyle ilişkisini gözetir; her piksel yanındaki pikselle karşılaştırılır ve büyük-küçük durumuna göre 0 ya da 1 yazılır.

Kare farkı (inter-frame differencing) klasik bir yöntemdir: ardışık iki kare arasındaki piksel farkına bakılır. Makalenin önerdiği geliştirilmiş üç kare farkı, algısal hash yöntemlerini kullanarak yalnız ardışık iki kare arasındaki farka değil, birinci ve üçüncü kare arasındaki farka da bakar; yani fark penceresini genişletir. Yol üzerinde hareket eden araçların olduğu örnekte klasik yöntem bir önceki kareyle kıyasladığı için küçük bir değişim görüyordu; geliştirilmiş yöntem iki önceki kareyle de kıyasladığı için hareketin doğrultusunu tahmin edebilecek duruma geliyordu.

Nesne takibinde ilk karedeki kişi bir kaynak görüntü (source image) olarak alınır. Kişi hareket ettikçe algısal hash'ler arasındaki farkın en az olduğu bölgeye bir pencere çizilir. aHash yalnız ortalamaya göre kıyasladığı için pencerede kayma oldu; pHash ve dHash daha tutarlı sonuç verdi.

3.4 Dördüncü Makale: LHash ve LDHash

Aynı ekip iki yıl sonra iki yeni algısal hash yöntemiyle geri geldi; daha doğru, daha dayanıklı ve daha az maliyetli olduklarını iddia ettiler. Yine görsel takip (visual tracking) yaptılar.

- **LHash (Laplace tabanlı hash):** Girdi görüntüsü gri seviyeye çevrilir, boyutu küçültülür, ardından Laplace dönüşümü uygulanır. Laplace dönüşümü köşe ve geçiş bölgelerindeki bilgiyi güçlendirir; slayttaki örnekte bir gölgenin oluşturduğu köşe, Laplace sonrası daha belirgin görünüyordu. Sonra ikilileştirme (binarization) yapılıp ortalamayla karşılaştırılarak hash elde edilir.
- **LDHash:** LHash'ten farklı olarak araya bir gradyan görüntüsü hesaplama adımı ekler; görüntünün gradyanı alınır, ardından aynı adımlarla (Laplace, ikilileştirme, ortalamayla kıyas) sıfır ve birlerden oluşan hash elde edilir.

Farklı videolar üzerindeki takip başarı oranı (tracking success rate) tablosunda LDHash çoğunlukla diğer yöntemlerden daha iyi çalıştı; yalnız bir videoda başka bir yöntem öne geçti. Yüz takibi örneğinde başka bir yöntem yanlış bir bölgeyi gösterirken önerilen yöntem yüzü bütün karelerde buldu. Algısal hash burada akan video karelerinde kaynak yüz görseline en az farkla benzeyen bölgeyi bulmaya yarıyordu.

Grubun özeti: makalelerin hepsi algısal hash'in karşılaştırma, fark bulma, değişiklik takibi, hareket takibi ve nesne tespiti gibi birçok alanda kullanılabileceğini gösteriyor. Gelecek çalışmalarda sosyal medya içerik tanımlama ile yüz ve nesne tanımada önemli rol oynaması bekleniyor. Hoca konunun farklı ve ilgi çekici olduğunu, makalelerden görüntü örnekleri konmasının açıklayıcı olduğunu söyledi.

4. Blokzincirde Hash Fonksiyonları ve Proof of Work (Dördüncü Sunum)

4.1 Hash Fonksiyonunun Özellikleri

Hash fonksiyonu, keyfi uzunlukta bir girdiyi bir fonksiyondan geçirip sabit uzunlukta (n bit) bir çıktı veren matematiksel bir algoritmadır (ekrandan, 00:50:00). Veri erişimi, şifreleme ve veri bütünlüğü için kritiktir. Sunumda sayılan özellikler:

- Aynı girdi her zaman aynı çıktıyı verir.
- Hesaplama açısından verimlidir; çok büyük veri setinde de hızlı çalışır, ama algoritmalar arasında hız farkı vardır.
- Ön görüntü direnci: üretilmiş bir hash'ten girdiye geri dönme olasılığı çok düşük ya da imkansızdır.

- Çakışmasızlık (collision free): iki girdi aynı çıktıyı vermemelidir. Öğrenci bunu en önemli özelliklerden biri saydı.

4.2 Bitcoin ve Merkeziyetsiz Açık Defter

Bitcoin'i en iyi tarif eden kavram merkeziyetsiz açık defterdir (decentralized public ledger). Defter, "A kişisi B'ye 50 Bitcoin ödedi" gibi bütün işlemlerin alt alta tutulduğu bir hesap defteri gibi düşünülebilir. Herkese açıktır; merkeziyetsiz olduğu için arada bir aracı yoktur ve ağdaki her düğüm deftere erişebilir. Bütün işlemler elde olduğu için bir kişinin elinde son durumda ne kadar para kaldığı hesaplanabilir. Öğrenci bu yüzden açık defterin kendisinin Bitcoin olduğunu söyledi: bütün paraların hesabı buradan çıkarılır.

İşlemler bloklara bölünür ve her blok önceki bloğun hash'ini tuttuğu için yapıya blokzincir denir. Her blok kendi işlem geçmişini taşır; ayrıca her blokta Proof of Work denen özel bir sayı vardır. Bloklar değiştirilemez (immutable): her blok bir önceki bloğun hash'ini kullandığı için aradaki bir işlem değiştirilirse ondan sonraki bütün blokların hash'i değişir ve değişiklik algılanır.

4.3 Dijital İmza ile Tekrarlanan İşlemlerin Önlenmesi

Defter açık olduğu için herkes bir şey ekleyebilir. Örneğin bir kişi birine 20 Bitcoin gönderdiğini bildiren işlemi deftere bir değil beş ya da yirmi kez ekleyebilir. Bunu önlemek için dijital imza kullanılır. Dijital imza kendi içinde hash fonksiyonu ve şifreleme kullanır: kişi işlemini doğrulamak için onu kendi özel anahtarıyla imzalar, imzalanan hash bloğa eklenir; imza çözülerek doğrulanıp doğrulanmadığı kontrol edilir. Her işlemin benzersiz bir kimliği (unique ID) vardır ve bu kimlik de imzalanır; böylece her işlemin benzersiz olduğu kontrol edilebilir. Slayttaki şemada her işlem bir sonraki sahibin açık anahtarı ve önceki işlemin hash'iyle birlikte bir önceki sahibin özel anahtarıyla imzalanıyor, imza önceki sahibin açık anahtarıyla doğrulanıyordu (ekrandan, 01:00:00).

4.4 SHA-256

Blokzincirde kullanılan örnek algoritmalar listelendi (Bitcoin'de SHA-256; Ethereum ve Litecoin'de başka algoritmalar). Sunumun ağırlığı SHA-256 üzerindeydi. Adı 256 bitlik sabit çıktısından gelir; NSA tarafından tasarlanmıştır. Genel işleyişi:

1. Veri 512 bitlik parçalara bölünür. 512 bitten kısa kalan parçanın sonuna bir adet 1 biti eklenir, kalan bitler 0 ile doldurulur; böylece her parça 512 bit olur.
2. Başlangıç hash değerleri ayarlanır: ilk 8 asal sayının kareköklerinin kesirli kısımlarından alınan sekiz değer.
3. Her 512 bitlik blok 32 bitlik kelimelere bölünür ve bu sekiz değerle ana döngüde (main loop) çeşitli fonksiyonlarla işlenir.
4. Sonuçlar birleştirilerek 256 bitlik çıktı elde edilir.

SHA-256 güçlü ve hızlı olduğu için yaygın kullanılır; Bitcoin'de işlemlerin içinde de kullanılır.

4.5 Proof of Work ve En Uzun Zincir

Defter herkese açık olduğu için düğümler arasında doğru işlem geçmişinin hangisi olduğu konusunda bir uzlaşı (konsensüs) gerekir; Proof of Work bunun için ortaya atılmıştır. Ağ ya da algoritma, blok hash'inin baştaki belirli sayıda basamağının sıfır olmasını şart koşar. Madenci (miner) düğümler bloğun sonuna özel bir sayı ekleyip bütün bloğu hash'ler; hash istenen sayıda sıfırla başlayana kadar bu sayıyı değiştirir. İlk bulan bloğu doğrulamış olur, bulduğu sayıyı bloğa ekleyip ağa yayınlar. Doğrulamak kolaydır: hash fonksiyonu hızlı çalıştığı için herkes gelen değeri hash fonksiyonuna koyup şartın sağlandığını hemen kontrol eder. Zincirin güvenliği böyle sağlanır.

Ağ eşler arası (peer-to-peer) olduğu için birden fazla işlem geçmişi yayınlanabilir. Hangisinin doğru kabul edileceği için her düğüm her zaman en uzun doğrulanmış zinciri alır. Bazı düğümler geride kalsa bile yeni doğrulanmış bloklar yayıldıkça herkes bir noktada birbirine yetişir. Hash fonksiyonları ayrıca işlem zamanlarının kontrolünde (zaman sunucusu, timestamp server) da kullanılır.

5. Evrensel Hash Fonksiyonları (Beşinci Sunum)

Bu sunum literatür taramasını ağırlıklı olarak evrensel hash'in matematiksel tanımları, teoremleri ve ispatları üzerine kurdu.

5.1 Motivasyon

Sunumda evrensel hash'in 1978'de iki bilim insanı tarafından ortaya atıldığı söylendi. Ortaya çıkış nedeni şudur: bir algoritma geliştirildiğinde hangi problemlerde ya da uygulamalarda kullanılacağı bilinmez; bu yüzden o algoritma için en kötü girdi karşısında performans düşük olabilir. Performansı iyileştirmek ve çakışmaları azaltmak için tek bir hash fonksiyonu yerine bir hash fonksiyonları ailesi tanımlanır ve aralarından rastgele bir fonksiyon seçilerek kullanılır. İki özellik gerekir:

- **Rastgele seçim:** Ailede çok sayıda fonksiyon olmalı ve aralarından rastgele seçim yapılabilmelidir.
- **Çeşitlilik:** Farklı anahtarlar farklı çıktılar vermeli, yani çakışma azaltılmalıdır.

  Ders sonrası not: Sunumda evrensel hash'i ortaya atanlar "Beckman ve Carter, 1978" diye anıldı. Büyük olasılıkla doğrusu J. L. Carter ve M. N. Wegman'dır; çalışma 1977'de konferansta, 1979'da dergide yayımlanmıştır.

5.2 Tanım

Slayttaki tanım (ekrandan, 01:06:40): Hash fonksiyonu U adındaki evrensel anahtar kümesini $[0, m-1]$ aralığındaki daha küçük bir kümeye eşler, $h : U \to \{0, 1, \dots, m-1\}$. Birbirinden farklı iki tam sayı için çakışma göstergesi

$$U(x,y) = \begin{cases} 1, & x \neq y \text{ ve } h(x) = h(y) \\ 0, & \text{diğer durumlarda} \end{cases}$$

biçimindedir. H ailesinden rastgele seçilen h için birbirine eşit olmayan herhangi iki anahtarın çıktılarının eşit olma olasılığı $1/m$'den büyük değilse H evrenseldir:

$$\Pr_{h \leftarrow H}[h(x) = h(y)] \leq \frac{1}{m}.$$

Burada m slot sayısıdır. Çıktılardaki çakışma ne kadar azsa hash fonksiyonu o kadar kullanışlıdır.

5.3 Beklenen Çakışma Sayısı

n anahtar m slotlu bir tabloya hash'lendiğinde, bir x anahtarı için beklenen çakışma sayısı n/m'den küçük olmalıdır, çünkü en kötü durumda bile tabloya n anahtar konmuştur. İspat (ekrandan, 01:10:00):

H evrensel bir aile, $x, y \in U$ rastgele anahtarlar olsun. C_x, x ile çakışan anahtarların toplam sayısıdır. Rastgele seçilen h altında $h(x) = h(y)$ ise $c_{xy} = 1$, değilse $c_{xy} = 0$'dır. Buna göre:

$$C_x = \sum_{y \neq x} c_{xy}$$

$$E[C_x] = E\Big[\sum_{y \neq x} c_{xy}\Big] = \sum_{y \neq x} E[c_{xy}] = \sum_{y \neq x} \frac{1}{m} = \frac{n-1}{m}.$$

İkinci eşitlik beklenen değerin doğrusallığıdır. Üçüncü eşitlikte c_{xy} 0 ya da 1 değerini aldığı için beklenen değeri $1 \cdot \Pr[c_{xy} = 1] + 0 \cdot \Pr[c_{xy} = 0] = 1/m$ olur (evrensellik tanımından en fazla $1/m$). x dışında $n-1$ anahtar vardır. Sonuç $(n-1)/m < n/m$ olduğu için iddia ispatlanmış olur.

5.4 Asal Sayı (Prime) Yöntemi

Asal sayı yönteminde anahtarlar bir asal sayıya göre modüler bölünerek hash değeri elde edilir. Asal sayı genellikle hash tablosunun boyutuna yakın seçilir; yöntem çoğunlukla açık adresleme kullanan hash tablolarında kullanılır. Çakışmalar için etkili bir çözüm sunabilir, ama fonksiyonun ve asal sayının uygun seçilmesi önemlidir. Sunumda dezavantaj olarak asal sayı hesaplamanın zaman alması ve yerel çakışma olasılığının artabilmesi sayıldı.

Slayttaki tanım (ekrandan, 01:11:50): U'nun elemanları $\{0, \dots, U-1\}$ aralığında negatif olmayan tam sayılar olsun. $p \geq |U|$ olan bir asal sayı seçilir. a ve b belirtilen aralıklarda rastgele seçilmek üzere aile şöyle tanımlanır:

$$h_{a,b}(x) = ((ax + b) \bmod p) \bmod m, \qquad a \in \{1, \dots, p-1\},\ b \in \{0, 1, \dots, p-1\}.$$

a için $p-1$, b için p seçenek olduğundan ailedeki fonksiyon sayısı $|H| = p(p-1)$'dir. Evrensellik için bir $x_i \neq x_j$ çiftini çakıştıran fonksiyon sayısının en fazla $p(p-1)/m$ olması gerekir. Bu sayı $|H| = p(p-1)$'e bölünürse

$$\Pr[h(x_i) = h(x_j)] \leq \frac{p(p-1)/m}{p(p-1)} = \frac{1}{m}$$

elde edilir.

> **Ders sonrası not:** Slaytta en büyük değerler "$a = p-1$, $b = p$" diye yazılmış. b'nin alabileceği en büyük değer $p-1$'dir; $|H| = p(p-1)$ sonucu b'nin p farklı değer alabilmesinden gelir.

Neden $a \neq 0$? İki anahtar $x_i \neq x_j$ için $y_i = (ax_i + b) \bmod p$ ve $y_j = (ax_j + b) \bmod p$ yazılıp a ve b yalnız bırakıldığında, $a = 0$ durumunda $y_i = y_j = b$ olur. Yani hangi anahtar verilirse verilsin sonuç b çıkar ve bütün anahtarlar çakışır. Bu yüzden a sıfırdan farklı seçilir.

5.5 Matris Yöntemi

Matris yönteminde hash değeri, girdinin bir matrisle işleme sokulmasıyla elde edilir; matris genellikle tablo boyutuyla ilişkilidir. Büyük veri kümeleri için etkili bir tasarım sunabilir, ama matrisin seçimi ve veri dağılımı önemlidir.

Slayttaki tanım (ekrandan, 01:16:10): Anahtarlar u bit uzunluğundadır, yani $|U| = 2^u$. Tablo boyutu M 2'nin kuvvetidir; indeks b bit uzunluğundaysa $M = 2^b$ olur. Rastgele bir $b \times u$ boyutlu ikili h matrisi seçilir ve

$$h(x) = h\,x \pmod 2$$

tanımlanır; yani hash değeri, h matrisi ile x'in bit vektörü çarpılıp 2 tabanında mod alınarak bulunur. Çakışma olasılığı

$$\Pr[h(x) = h(y)] = \frac{1}{m} = \frac{1}{2^b}$$

olduğundan yöntem evrenseldir.

İspatın sezgisi şöyle anlatıldı: $x \neq y$ ise iki anahtar en az bir bitte ayrılır; hepsinin farklı olması gerekmez. Öğrenci gösterimde bu biti x için 0, y için 1 seçti. x'in o biti 0 olduğu için matrisin o bite karşılık gelen sütunu hx çarpımına hiçbir katkı yapmaz; sütun ne olursa olsun sonuç aynıdır. y'nin o biti 1 olduğu için aynı sütun hy sonucunu değiştirir. Matrisin geri kalanı sabit tutulup yalnız bu sütun rastgele seçildiğinde, $h(x) = h(y)$ eşitliğini sağlayan tek bir sütun değeri vardır; 2^b olası sütun arasından bu birinin seçilme olasılığı $1/2^b$'dir.

5.6 Polinom Yöntemi

Polinom evrensel hash, farklı x ve y anahtarlarını polinom değerleriyle hash'ler (ekrandan, 01:18:45). Hash tablosunun boyutuna bakılarak uygun polinom derecesi seçilir. b katsayısı sabit, x_i'ler girdinin karakterleridir; katsayının rastgele seçilmesi hash değerlerinin tabloya daha iyi yayılmasını sağlar:

$$H(x) = \left(x_0 + x_1 b + x_2 b^2 + \cdots + x_{n-1} b^{n-1}\right) \bmod M.$$

$x \neq y$ iken $H(x) = H(y)$ oluşuyorsa iki hash birbirinden çıkarıldığında sonuç M'ye tam bölünmelidir:

$$(x_0 - y_0) + (x_1 - y_1)b + (x_2 - y_2)b^2 + \cdots + (x_{n-1} - y_{n-1})b^{n-1} \equiv 0 \pmod{M}.$$

Ek bilgi (derste anlatılmadı): M asal ve b rastgele seçilirse, bu eşitlik b cinsinden derecesi en fazla $n-1$ olan sıfırdan farklı bir polinomun köküdür. Böyle bir polinomun $\mathbb{Z}_M$'de en fazla $n-1$ kökü olduğu için çakışma olasılığı en fazla $(n-1)/M$'dir. Bu yüzden polinom hash tam olarak $1/M$ değil, uzunlukla büyüyen bir sınırla "neredeyse evrensel" sayılır.

5.7 Hocanın Yorumu

Hoca sunumu dinlerken kendini bir derste gibi hissettiğini, teorik anlatımın güzel olduğunu söyledi. Geri bildirim olarak güncel çalışmalardan da söz edilmesini beklediğini belirtti. Öğrenci literatür taramasında matematiksel kısımlara odaklandığını, oradan çıkamadığını kabul etti.

6. Çakışmaya Dayanıklı Hash Fonksiyonları: Tasarım ve Tarihçe (Altıncı Sunum)

İki kişilik bir grup çakışmaya dayanıklı hash fonksiyonlarını (CRHF) anlattı.

6.1 Kriptografik Hash ve Sınıflandırma

Kriptografik hash fonksiyonu değişken boyuttaki bir girdiyi sabit boyutta bir çıktıya eşler; günümüzde çıktılar genellikle 128 ile 512 bit arasındadır. Slaytta bir `cat.jpg` dosyasının hash'i örnek gösterildi.

Hash fonksiyonları aldıkları parametrelere göre ikiye ayrılır:

- **Anahtarlı (keyed):** Gizli bir anahtarla çalışır. Ana alt kategorisi mesaj doğrulama kodlarıdır (MAC); gönderici ile alıcı arasındaki mesajın doğruluğunu sağlar.
- **Anahtarsız (unkeyed):** Kriptografik hash fonksiyonları olarak da geçer. Değişiklik tespit kodları (modification detection codes, MDC) ve diğer uygulamalar olarak ayrılır. MDC de ikiye ayrılır; grubun incelediği dal çakışmaya dayanıklı hash fonksiyonlarıdır.

6.2 Çakışma Tanımı ve Koşullar

$x \neq x'$ olmak üzere iki farklı girdi aynı hash'i üretirse buna çakışma denir. Herhangi bir olasılıksal polinom zamanlı algoritma için bir çakışma bulmak ne kadar imkânsızsa fonksiyon o kadar çakışmaya dayanıklıdır. H sonsuz girdi alanından sonlu çıktı aralığına bir sıkıştırma fonksiyonudur, bu yüzden çakışma teoride mutlaka vardır; ama çakışma olasılığı ne kadar düşükse fonksiyon o kadar güvenilirdir.

Çakışmaya dayanıklı bir hash fonksiyonunun sağlaması gereken koşullar:

- Sınırsız uzunluktaki girdiyi sabit uzunlukta bir çıktıya dönüştürmelidir.
- Tek yönlü olmalı, yani ön görüntü ve ikinci ön görüntü dirençlerine sahip olmalıdır.
- İki farklı girdi için aynı hash değerini vermemelidir.
- Doğum günü saldırısına göre, n bitlik bir hash için $2^{n/2}$'den az işlemle çakışma bulunamamalıdır.

Kullanım alanları: dijital imza, şifre depolama, bütünlük doğrulama. Slayttaki örnekte bir e-posta ekinin doğrulanması gösterildi: gönderen dosyanın hash'ini alıp karşı tarafa iletir, alıcı da dosyanın hash'ini alıp karşılaştırır; eşitse dosya değişmemiştir.

6.3 Temel Tasarım Yapıları

Yaygın kullanılan CRHF'ler iki yapıdan birine sahiptir:

- **Merkle-Damgård yapısı:** Girdi bloklara bölünür ve nihai değeri oluşturmak için bir sıkıştırma fonksiyonu yinelemeli olarak uygulanır; son aşamada istenen uzunlukta bir çıktı üretilir.
- **Sünger (sponge) yapısı:** Girdi bir fonksiyondan geçirilerek sabit boyutta olmayan bir ara durum üretilir; ikinci aşamada bu durum istenen boyuta "sıkılarak" çıktı alınır.

6.4 Algoritmaların Tarihçesi

- **MD5 (1992):** 128 bitlik, Merkle-Damgård yapısına uygun bir hash fonksiyonu. Güvenlik açısından zayıf olduğu kanıtlanmıştır; farklı iki girdinin aynı hash'i ürettiği örnekler internette mevcuttur.
- **SHA-1 (1995):** 160 bitlik hash üretir, Merkle-Damgård yapısındadır (ekrandan, 01:30:00). Slayttaki şemada 512 bitlik mesaj bloğu, başlatma vektörleri, A-E beş değişkeni ve dört tur grubu (0-19, 20-39, 40-59 ve 60-79) ile 160 bitlik hash gösteriliyordu. SHA-1 özünde MD4'e dayanır. Aynı hash'i üreten farklı dosyalar bulunduğu için zayıf kabul edilir; Google'ın bir araştırmasında iki farklı PDF'in aynı SHA-1 değerini verdiği gösterilmiştir.
- **SHA-2:** NSA tarafından geliştirilip NIST tarafından yayımlanmıştır; 256 ve 512 bit çıktı üretebilir ve Merkle-Damgård yapısını kullanır. Henüz çakışmaya rastlanmamıştır, ama teorik olarak rastlanabileceği öngörülür.
- **SHA-3:** NIST'in düzenlediği yarışma sonucunda belirlendi; en güvenli kabul edilen Keccak algoritmasıdır. Merkle-Damgård yerine sünger yapısını kullanır ve farklı bit sayılarında hash üretebilir.

  Ders sonrası not: SHA-1 şemasındaki son tur grubu slaytta "Round 50-79" diye yazılmış (ekrandan, 01:30:00); 80 tur 20'şerlik dört grup olduğu için doğrusu 60-79'dur.

Grubun özeti: bu algoritmalar temelde MD4'ten gelir; ilk iki algoritmada (MD5 ve SHA-1) çakışma bulunmuştur. Sunum sonunda algoritmaların hız karşılaştırması da gösterildi.

7. Hash Fonksiyonlarının Ataklara Dayanıklılığı (Yedinci Sunum)

İki kişilik bir grup, hash fonksiyonlarının saldırılara karşı dayanıklılığını okudukları makaleler üzerinden karşılaştırdı.

7.1 Temel Özellikler ve Doğum Günü Paradoksu

Hash fonksiyonu farklı uzunluktaki girdileri alıp sabit bir çıktı üreten bir dizi matematiksel işlemdir. Parolaların veritabanında saklanması, dosyada değişiklik yapılıp yapılmadığının kontrolü, elektronik imza ve blokzincir gibi kritik alanlarda kullanıldığı için iyi ve kötü hash fonksiyonunu ayıran özellikler önemlidir.

- **Çakışma:** İki farklı girdi için aynı değerin üretilmemesi beklenir. Girdi uzayı sonsuz, çıktı uzayı sonlu olduğu için bu teorik olarak tam mümkün değildir; ama olabildiğince azaltılmalıdır. Buna en çok verilen örnek doğum günü paradoksudur: 23 kişinin bulunduğu bir grupta iki kişinin doğum gününün aynı olma olasılığı yaklaşık %50'dir; 40-50 kişide bu olasılık %90'ın üzerine çıkar.
- **Ön görüntü:** Üretilen hash'ten asıl değer saldırgan tarafından elde edilememelidir. Hocanın derste belirttiği gibi hash fonksiyonu tek yönlü olmalıdır: girdiden hash'e dönüşüm kolay, hızlı ve optimize; hash'ten girdiye dönüş olabildiğince zor olmalıdır.
- **İkinci ön görüntü:** Çakışma ile ön görüntünün birleşimi gibidir; verilen bir girdiyle aynı hash'i veren ikinci girdinin ne olduğunun bulunması zor olmalıdır.

Doğum günü olasılıkları $1 - \prod_{i=0}^{k-1} \frac{365-i}{365}$ formülüyle hesaplanınca $k = 23$ için 0,507, $k = 41$ için 0,903, $k = 50$ için 0,970 çıkar (python ile doğrulandı).

7.2 MD5

MD5, MIT profesörü Ron Rivest tarafından MD4'ün yerine geliştirildi. İnternetin ilk yıllarında ihtiyaçları karşıladığı için yaygın kullanıldı, ama günümüzde güvenli sayılmaz. Çıktısı 128 bittir; sunumda araştırmalara göre güvenli bir hash çıktısının en az 160 bit olması gerektiği söylendi.

Giriş mesajı 512 bitlik bloklara, her blok 16 adet 32 bitlik kelimeye ayrılır. 512 bite tamamlanmıyorsa sonuna bir adet 1 eklenip kalanı sıfırla doldurulur. Slayttaki anlatım (ekrandan, 01:40:00): ana algoritma A, B, C, D adlı dört adet 32 bitlik kelimeye ayrılmış 128 bitlik bir durum üzerinde çalışır; bunlar sabit değerlerle başlatılır. Her 512 bitlik mesaj bloğu bu durumu değiştirmek için kullanılır. Bir bloğun işlenmesi tur denilen dört benzer aşamadan oluşur; her tur doğrusal olmayan bir fonksiyon, modüler toplama ve bit düzeyinde sola döndürmeden oluşur. Dört olası fonksiyon vardır ve her turda farklısı kullanılır:

$$F(B, C, D) = (B \wedge C) \vee (\neg B \wedge D)$$
$$G(B, C, D) = (B \wedge D) \vee (C \wedge \neg D)$$
$$H(B, C, D) = B \oplus C \oplus D$$
$$I(B, C, D) = C \oplus (B \vee \neg D)$$

Burada $\oplus, \wedge, \vee, \neg$ sırasıyla XOR, AND, OR ve NOT işlemleridir. Şemada fonksiyon çıktısı A'ya eklenir, ardından mesaj kelimesi M_i ve sabit K_i eklenir, sola döndürülür ve B ile toplanır.

> **Ders sonrası not:** Sunumda MD5'in "93 yılında" geliştirildiği ve "16 tur" olduğu söylendi; slaytta da "Toplamda 16 tur vardır" yazıyor. MD5 büyük olasılıkla 1991'de tasarlanıp 1992'de RFC 1321 ile yayımlanmıştır (altıncı sunumda da 1992 dendi). Yapıda 4 tur ve her turda 16 adım, toplam 64 adım vardır.

7.3 SHA-1 ve SHA-2

SHA-1, NSA tarafından geliştirilen ve daha önce çıkan SHA-0'ın üzerinde küçük bir düzenleme yapılarak elde edilen bir fonksiyondur. MD5'in 128 bitine karşılık 160 bitlik özet üretir; çıktı 40 haneli onaltılık bir sayıdır. Şemada A, B, C, D, E adlı beş adet 32 bitlik değişken bir fonksiyona girer, sola kaydırılır ve her işlemde W_t ve K_t değerleri eklenir. 2005'teki bir kriptanaliz saldırısıyla SHA-1'in kullanım için yeterince güvenli olmadığı gösterildi; 2010'dan beri yerine SHA-2 ve SHA-3 önerilir. Microsoft, Google, Apple ve Mozilla SSL sertifikalarında SHA-1 desteğini çekeceklerini açıkladı.

SHA-2 tek bir fonksiyon değil, bir ailedir. En yaygını SHA-256'dır; yanında SHA-384 ve SHA-512 de vardır. Yeni sürümler SHA-1'in zayıflıklarını taşımadan onunla aynı yapıyı, modüler aritmetiği ve mantıksal ikili işlemleri kullanır. SHA-256 sekiz adet 32 bitlik kelime üzerinde, SHA-512 sekiz adet 64 bitlik kelime üzerinde çalışır; temel tasarım aynıdır. SHA-384, SHA-512'nin küçük bir değişikliğidir.

> **Ders sonrası not:** Sunumda SHA-256 ve SHA-512'nin "64 bit işlemciler için tasarlandığı" söylendi. 64 bitlik kelimelerle çalıştığı için 64 bit işlemcilerde avantajlı olan SHA-512'dir; SHA-256 32 bitlik kelimeler kullanır.

7.4 SHA-3 ve Saldırı Sonuçları

Grubun ikinci üyesi SHA-3'ten devam etti. SHA-2'nin de belirli sorunları görülünce daha iyisini geliştirmek için bir yarışma düzenlendi. Beş finalist vardı; literatür tarandığında Keccak'ın öne çıktığı görülür. Sünger yapısında emme (absorb) ve sıkma (squeeze) aşamalarında verinin bir kısmı fonksiyonlardan geçirilir, diğer kısmı olduğu gibi devam eder; böylece çıktıda rastgele görünen sonuçlar elde edilir.

Literatür taramasında iki tür çalışma görüldü: fonksiyonların matematiksel olarak ne kadar dayanabildiğini hesaplayanlar ve algoritmayı inceleyerek daha hızlı çakışma üretmeye çalışanlar.

Teorik dayanıklılık tablosu (ekrandan, 01:45:40):

Fonksiyon	Çıktı	İç durum	Blok	Tur	Çakışma	Ön görüntü	2. ön görüntü
SHA-1	160	160 (5×32)	512	80	<80	162	160-L(M)
SHA-224	224	256 (8×32)	512	64	112	224	min(224, 256-L(M))
SHA-256	256	256 (8×32)	512	64	128	256	256-L(M)
SHA-384	384	512 (8×64)	1024	80	192	384	384
SHA-512	512	512 (8×64)	1024	80	256	512	512-L(M)
SHA3-256	256	1600 (5×5×64)	1088	24	128	256	256
SHA3-512	512	1600 (5×5×64)	576	24	256	512	512

Tabloda ayrıca SHA-512/224, SHA-512/256, SHA3-224 (blok 1152), SHA3-384 (blok 832) ve SHAKE128/SHAKE256 (blok 1344/1088, güvenlik $\min(d/2, 128)$ ve $\min(d/2, 256)$) satırları vardı. Grubun ilgilendiği kısım en sağdaki güvenlik bitleriydi.

> **Ders sonrası not:** Sunumda SHA-1 için "<80" değeri, "80 bit değişiminden sonra çakışmalar oluşmaya başlıyor" diye yorumlandı. Güvenlik gücü (security strength) bit cinsinden bir işlem sayısıdır: SHA-1 için "<80", çakışmanın 2^{80}'den az işlemle bulunabildiği anlamına gelir. Değiştirilen bit sayısıyla ilgisi yoktur.

Literatür karşılaştırmasında RIPEMD-160, MD4 ve MD5'in geliştirilmiş bir sürümü olarak anıldı; bazı makalelerde daha düşük karmaşıklıkla saldırı yapılabildiği gösterilmiştir. SHA-1 için 2005'te yayımlanan bir çalışmaya göre azaltılmış 53 turluk sürümde çakışmalar bulunabilmektedir. SHA-2'nin farklı girdi-çıktı boyutlu sürümleri için de azaltılmış turlarda daha düşük karmaşıklıkla saldırılar gösterilmiştir. SHA-3 için pratik bir saldırı makalesi bulunamadı; tabloda yalnız teorik ön görüntü, ikinci ön görüntü ve sözde ön görüntü (pseudo-preimage) saldırılarının kaç tura kadar uygulanabildiği verildi.

İstatistiksel saldırılar: Girdinin yalnız bir biti değiştirilerek çıktıda kaç bitin değiştiği gözlemlenir. Değişen bit sayısı ve oranları ile bunların standart sapması incelenir; standart sapmadaki değişimin küçük olması çıktının tahmin edilmesini zorlaştırır. Sunumda SHA-1 ve SHA-256'da bu değişimin çok az olduğu söylendi. Bir grafikte mesaj boyutu arttıkça SHA-160, SHA-256, SHA-512 ve SHA-224 için güvenli bit sayısının nasıl değiştiği gösterildi.

7.5 Önerilen Güvenli Sürümler

Sunumun sonunda 2022'de yapılan bir çalışmadan alınan bir şema gösterildi (ekrandan, 01:50:00). Önerilen hash fonksiyonları ve dayandıkları çalışmalar:

- Geliştirilmiş MD5 algoritması (Guzman ve ark., 2018)
- SHA-1 için paralel algoritma (Kishore ve Kapoor, 2014)
- SHA-256 (Quist-Aphetsi ve Senkyire, 2019)
- SHA-3 (Easttom, 2021; Maetouq ve ark., 2018)
- RipeMD (Easttom, 2021)
- Whirlpool (Wu ve ark., 2015)

Sunumda 2018'deki geliştirilmiş MD5'in tercih edilmesinin daha faydalı olduğu söylendi.

8. Algısal Özetleme: aHash, dHash, pHash, wHash (Sekizinci Sunum)

Bu sunum algısal hash'i temel algoritmalar düzeyinde, giriş seviyesinden anlattı. Hoca sunum öncesinde, aynı konunun birçok kez anlatıldığına değinerek "çakışmaya dayanıklı olanlar çakıştı" diye şaka yaptı.

8.1 Kriptografik Hash'ten Algısal Hash'e

Klasik hash'te bir girdi (mesaj ya da düz metin) bir algoritmadan geçirilir ve bir hash değeri elde edilir. Teknolojik gelişmelerin dijital dünyaya yansıması 1990'larda yaygınlaştı ve hash işlemleri görüntülere de uygulanmaya başlandı. O yıllarda "algısal hash" terimi pek kullanılmasa da bu yaklaşımın temelini oluşturan matematiksel ifadeler ortaya çıktı. Ardından verilerin saklanması, korunması, telif hakkı ve dijital parmak izi gibi konular gündeme geldi.

Algısal hash'te hedef, farklı ama algısal olarak benzer resimlerden benzer özet değerleri üretmektir. İnsan gözü bir resmin renkli, diğerinin gri olduğunu görse de içlerinde aynı nesneler varsa onları aynı algılar; burada renkler önemsizdir. Algısal hash bunu matematiksel bir modele dönüştürmeye çalışır. Slayttaki örnekte bir resim ve onun farklı ölçek, kontrast ve renklerdeki üç kopyası vardı. Kriptografik hash'te en küçük değişiklik çıktıya büyük yansır; algısal hash'te ise bu dört resmin hash'lerinin belirli oranda benzer olması istenir. Hepsinde aynı yapı (örnekte Galata) tanınabildiği için algısal bir bütünlük vardır.

8.2 Benzerlik Kararı

İki resim önce bazı ön işlemlerden geçirilir, sonra hash'leri hesaplanır ve bağlama göre belirlenen eşik değerlere göre algısal benzerlik derecesi bulunur. Bir referans görüntü ile rengi değiştirilmiş bir test görüntüsü örneğinde iki 8×8'lik hash elde edildi; benzerlik farklı uzaklık metrikleriyle hesaplanabilir. Hamming uzaklığıyla iki hash arasında farklı olan bit sayısı bulunur; örnekte bu 9 idi.

Genel tanım: iki görüntü I_1 ve I_2 bir algısal hash fonksiyonundan geçirilerek H_1 ve H_2 elde edilir. Bir uzaklık fonksiyonuyla H_1 ve H_2'nin benzerliği hesaplanır. Uzaklık bir σ_1 eşiğinin altında kalıyorsa görüntüler "algısal olarak benzer", σ_2 eşiğinin üstündeyse "algısal olarak benzer değil" kabul edilir. Eşiklerin seçimi üzerine çok çalışma vardır; özetle bağlama göre seçilirler.

8.3 Algısal Hash'in Özellikleri

- **Düşük çakışma olasılığı:** Algısal olarak farklı her veri için farklı hash üretilmeli, çakışma olasılığı düşük olmalıdır.
- **Kompaktlık:** Girdi boyutu ne olursa olsun çıktı sabit uzunlukta olmalıdır; bu, büyük görüntü veritabanlarında aramada verimli indeksleme sağlar.
- **Verimlilik:** Gerçek zamanlı ve büyük ölçekli veritabanlarında hızlı hesaplanabilmelidir.
- **Anlamsal değişikliklere düşük duyarlılık:** Kriptografik hash'teki çığ etkisi burada olmamalıdır.
- **Ayırt edicilik ve dayanıklılık:** Algısal bütünlük korunduğu sürece varyasyonlara karşı hash önemli ölçüde değişmemeli; algısal bütünlük değişiyorsa hash de belirgin biçimde değişmelidir.

Kullanım alanları: içerik tanıma ve telif hakkı koruma (içeriğin özgün olup olmadığı, kopyalanıp değiştirilip değiştirilmediği), adli görüntü analizi (görüntüde oynama olup olmadığı), büyük görüntü veritabanlarında benzer görüntü arama.

8.4 Genel İş Akışı

Slayttaki şema (ekrandan, 02:00:00):

```
Görüntü seti -> Ön işleme (yeniden boyutlandırma, filtreleme, aydınlatma düzeltme,
                           boyut indirgeme, renk uzayı dönüşümü, ...)
             -> Hash üretimi (özellik çıkarma, son işleme)
             -> Görüntü hash'leri
             -> Karar verme (referans hash'lerle; Öklid uzaklığı, Hamming uzaklığı,
```

```
                    L1 normu, korelasyon katsayısı, ...)
        -> Algısal benzerlik sonuçları
```

Aynı slayttaki ön işleme örneğinde orijinal görüntü sırasıyla gri seviyeye çevrilmiş, 32×32'ye küçültülmüş ve 7×7'lik bir çekirdek (kernel) uygulanmıştı.

Literatürdeki yöntemler matematiksel yöntemler ve derin öğrenmeye dayalı yöntemler olarak ikiye ayrılır; temel çalışma adımları ortaktır:

1. Girdi medyadan algısal olarak ayırt edici özellikler çıkarılır (örneğin doku ya da kenar).
2. Bu özelliklerin nasıl temsil edileceği belirlenir (boyut indirgeme, gri seviyeye dönüştürme gibi).
3. Elde edilen sayısal temsil bir özet fonksiyonundan geçirilir ve görüntüye özgü bir algısal hash elde edilir.
4. Hash'ler farklı uzaklık ölçüleriyle karşılaştırılıp algısal olarak benzer olup olmadıklarına karar verilir.

Basitleştirilmiş bir örnekte görüntü gri seviyeye çevrilip 5×5 bloğa bölündü. Her bloğun ortalaması resmin genel ortalamasıyla karşılaştırıldı; büyükse 1, değilse 0 yazıldı. Bitler satır satır art arda eklenerek hash değeri oluşturuldu.

8.5 Temel Algoritmalar

aHash (average hash). En kolay yöntemdir. Görüntü verisini alır ve 64 bitlik bir özet üretir. Görüntü gri seviyeye çevrilir, 8×8'e küçültülür, 64 hücrenin ortalaması alınır. Her hücre ortalamayla karşılaştırılır: büyükse 1, küçükse 0 yazılır.

dHash (difference hash). aHash'e çok benzer, ama ortalamaya bakmak genel bir yaklaşım olduğu için pikseller arasındaki farklara bakar. Görüntü gri seviyeye çevrilir, piksel bazında farklar alınır ve buna göre 1 ve 0 yazılır. Basitliği ve verimliliği nedeniyle benzerlik analizlerinde sık kullanılır.

Slayttaki bir uygulamada bir girdi görüntüsü, onun bozulmuş ya da gri seviyeye çevrilmiş hâlleri ve tamamen farklı bir görüntü karşılaştırıldı. 1 ve 4 numaralı görüntüler aynı renk tonlarına sahip ama bambaşka resimlerdi; algısal bütünlük olmadığı için az benzer çıkmaları gerekirdi. dHash bunları daha az benzer, aHash daha fazla benzer buldu; öğrenci buradan aHash'in algısal ayırt etme gücünün daha az olduğu sonucunu çıkardı. Ortalama filtresi uygulanmış görüntüde aHash'in %95 benzerlik vermesi de aHash'in zaten ortalamaya dayanmasıyla açıklandı.

pHash. Bu kez görüntü 32×32'ye küçültülür ve gri seviyeye çevrilir. Basit işlemler yerine ayrık kosinüs dönüşümü (DCT) uygulanır ve bir katsayılar matrisi elde edilir. Yüksek frekanslar atılır, düşük frekans katsayıları işleme alınır. Sonra yine büyük-küçük karşılaştırmasıyla 1 ve 0 yazılır ve satır satır art arda eklenerek pHash oluşur. Öğrenciye göre 32×32 daha küçük bloklar ve daha dayanıklı sonuç demektir. pHash içerik tabanlı sistemlerde benzerlik karşılaştırmasında sık kullanılır ve dayanıklı bir yöntem olarak bilinir.

wHash. pHash ile aynı mantıktadır, yalnız dönüşüm farklıdır (ekrandan, 02:10:00). Girdi gri seviyeye çevrilir, ayrık dalgacık dönüşümü (DWT) uygulanır. Dönüşüm görüntüyü farklı ölçek ve yönleri temsil eden katsayılara ayırır; algısal öneme göre katsayılar seçilir. Seçilen katsayıların ortalaması alınır, her katsayı ortalamadan büyük ya da eşitse 1, küçükse 0 atanır ve bitler art arda eklenir. Çok çözünürlüklü temsil sayesinde farklı ölçek ve çözünürlükteki görüntülerin karşılaştırılmasında öne çıkar.

8.6 Karşılaştırmalı Bir Çalışma

Bir çalışmada bir veritabanından 800 farklı görüntü alınmış ve her birine 10 farklı görsel manipülasyon uygulanmıştı; hash'lendikten sonraki benzerlikleri karşılaştırılmıştı. Çakışma olasılıkları grafiğe döküldüğünde bir μ ve σ değerine (beklenen dağılıma) yakınsayıp yakınsamadıklarına bakıldı. pHash ve dHash beklenen dağılıma daha yakın, aHash ve wHash daha uzak çıktı. Çalışma bunun çakışma olasılığı açısından ayırt etme yeteneğinin daha az doğru olmasına yol açabileceğini belirtiyordu; farklı eşik değerleri için verilen sonuçlar da bunu destekliyordu.

Öğrencinin yorumu: içerik tabanlı görüntü almada pHash yaygındır, ama dönüşüm işlemleri matematiksel bir maliyet getirir; pHash ve wHash bu açıdan maliyetlidir. Adli görüntü analizi gibi çok hassas işlerde daha hassas algoritmalar, hız gereken yerde dHash kullanılabilir.

9. Blokzincir Mimarisi ve Kullanılan Hash Fonksiyonları (Dokuzuncu Sunum)

Bu sunum blokzincir mimarisini ve blokzincirde kullanılan hash fonksiyonlarını genel düzeyde anlattı.

9.1 Hash Fonksiyonlarının Temel Özellikleri

Hash fonksiyonları verilerin benzersiz biçimde temsil edilmesini sağlayan kriptografik algoritmalardır; herhangi boyuttaki girdiyi sabit boyutta bir hash'e dönüştürürler. Hash girdinin bir özetidir ve tek yönlüdür; hash'ten girdi elde edilemez. Veri bütünlüğü doğrulama, parola kontrolü, güvenli veri depolama ve kimlik doğrulamada rol oynar. Sayılan özellikler:

- **Hızlı hesaplama:** İşlem süresini kısaltır ve blokzincir gibi veri işleme sistemlerinde performansı artırır.
- **Sabit boyut:** Depolama alanından tasarruf sağlar.
- **Benzersizlik:** İdeal bir hash fonksiyonunda iki farklı girdi aynı hash ile ifade edilmez.
- **Kolay hesaplama, zor tersine çevirme:** Hash'ten gerçek girdiye ulaşmak neredeyse imkansızdır.
- **Küçük değişikliklerde büyük farklılık:** Yapılan değişiklikler ya da saldırılar hızla fark edilir.

9.2 Blokzincirin Özellikleri ve Türleri

Blokzincir merkezi olmayan bir veritabanı teknolojisidir. Ardışık bloklardan oluşan bir zincirdir; her blok kendi verisini ve önceki bloğun hash değerini tutar. Merkezi otorite yoktur; sistemdeki her kullanıcı bir düğümdür ve her düğüm kayıt defterinin kendi kopyasına sahiptir. Kripto paralarla hayatımıza girmiş olsa da finans, sağlık ve tedarik zinciri kontrolü gibi alanlarda da kullanılır. Özellikleri:

- **Dağıtık yapı:** Veriler düğümler arasında paylaşılır, her düğüm diğerleriyle etkileşebilir.
- **Şeffaflık:** Her düğüm ağdaki tüm işlemleri görebilir ve denetleyebilir.
- **Değiştirilemezlik:** Veri bir kez kaydedildikten sonra geri alınamaz, güncellenemez, silinemez.
- **Bağımsızlık:** Mutabakat yapısı sayesinde merkezi bir sisteme gerek kalmadan veri güvenle aktarılır.
- **Kimlik gizliliği:** Düğümler kimlik belirtmeden, yalnız blokzincir adresiyle işlem yapabilir.

Blokzincir türleri (ekrandan, 02:20:00):

Tür	Özellikler
Açık	En yaygın türdür; herkes katılabilir ve katkıda bulunabilir; tamamen bağımsızdır; kripto paralarda kullanılır; internete benzetilebilir.
Özel	Sınırlı bir kullanıcı grubu ya da kuruluş kullanır; erişim ve katılım kısıtlıdır; üçüncü kişilerin bazı bilgilere erişimi engellenir; intranete benzetilebilir.
Hibrit	Açık ve özel blokzincirin özelliklerini birlikte taşır; gizlilik gereken işlemlerde özel özellikler kullanılır; veriler yalnız yetkilendirilmiş katılımcılarla paylaşılır.

Tür	Özellikler
Konsorsiyum	Hibrite benzer; bir grup işletme, kuruluş ya da kurumun ortak oluşturup yönettiği bir zincirdir; işlemler katılımcılar arasında bir uzlaşı mekanizmasıyla doğrulanır.

9.3 Blokzincir Mimarisi

Mimari dört bileşenden oluşur: blok, dijital imza, mutabakat (uzlaşı) mekanizması ve çatallaşma.

- **Blok:** Verilerin kaydedildiği temel birimdir. Sunumda her bloğun beş öğeden oluştuğu söylendi: veri, bloğun hash'i, önceki bloğun hash'i, zaman damgası ve nonce. Her blok önceki bloğun hash'ini tutarak zinciri oluşturur. Zaman damgası her bloğun ne zaman oluşturulduğunu ve zincirin akışını gösterir; değiştirilemezliği ve güvenilirliği artırır. Nonce, hash oluşturulurken kullanılan bir kerelik bir sayıdır.
- **Dijital imza:** Kullanıcıların kimliklerini kanıtlamalarını ve işlemlerini güvence altına almalarını sağlar. Gönderen mesajı özel anahtarıyla imzalar; alıcı imzalı mesajı açık (ortak) anahtarla doğrular.
- **Mutabakat mekanizması:** Dağıtık bir ağda birden fazla taraf arasında güven ve fikir birliği sağlar; işlemlerin doğrulanıp onaylanması ve zincire eklenmesi bu süreçle olur.
- **Çatallaşma (fork):** Sert çatallaşmada (hard fork) yapılan değişiklik önceki bloklarla uyumlu değildir; eski düğümlerin uyum sağlaması için yazılım güncellemesi gerekir ya da eski blokların geçerliliği ortadan kalkar. Yumuşak çatallaşmada (soft fork) değişiklik önceki bloklarla uyumludur; yeni kurallar eklenirken eskiler korunur ve eski ve yeni düğümler ağda birlikte çalışabilir.

  Ders sonrası not: Sunumda dijital imzada "herkesin iki özel anahtarı var" dendi. Her kullanıcının bir özel (private) ve bir açık (public) anahtarı vardır; imza özel anahtarla üretilir, açık anahtarla doğrulanır.

9.4 Blokzincirde Kullanılan Hash Fonksiyonları

- **SHA-256:** Bitcoin gibi popüler ağlarda yaygın kullanılır; 256 bitlik çıktı üretir. Blok verilerinin özetini hesaplamak, işlem doğrulama ve bütünlük için kullanılır.
- **Keccak-256:** 256 bit çıktı üretir; Ethereum ile birlikte, blokzincirde kodlama (akıllı sözleşmeler) yaygınlaşınca öne çıktı. Ethereum'un akıllı sözleşme platformunda ve hesap adreslerinde kullanılır.
- **BLAKE2 ve BLAKE3:** BLAKE2 genel amaçlı bir hash fonksiyonudur; yüksek hız ve güvenlik sunar, düşük bellekle çalışabilir, blok verilerinin ve işlemlerin özetlenmesinde kullanılır. BLAKE3, BLAKE2'nin eksiklerini tamamlamak için çıktı; daha hızlıdır ve paralel çalışabilir.
- **RIPEMD-160:** 160 bitlik çıktı üretir; Bitcoin gibi ağlarda adres oluşturma ve doğrulamada kullanılır.

  Ek bilgi (derste anlatılmadı): Bitcoin'de blok başlığının hash'i SHA-256'nın iki kez art arda uygulanmasıyla (double SHA-256) alınır. Adres üretiminde ise açık anahtarın önce SHA-256, sonra RIPEMD-160 özeti alınır; 160 bitlik adres böyle elde edilir.

10. Algısal Hash: Aşamalar ve Uygulama Alanları (Onuncu Sunum)

10.1 Tanım ve pHash

Algısal hash fonksiyonları, benzer görüntüler ya da sesler için aynı (ya da benzer) sonucu üretecek şekilde ayarlanır. İnsanın görmesini ve duymasını sağlayan özellik türlerine, renklere ve frekanslara odaklanarak insan algısını taklit eder. Diğer hash fonksiyonları en küçük değişikliğe karşı çok hassastır: örneğin bir pikseldeki mavi miktarını 200'den 199'a değiştirmek, yani tek bir biti çevirmek, hash bitlerinin yarısını değiştirebilir. Algısal hash'te ise küçük değişiklikler çıktıyı aşırı biçimde değiştirmez.

pHash, veriyi uzamsal alandan frekans alanına dönüştüren ayrık kosinüs dönüşümü üzerinden hash hesaplar; daha çok görüntüler için kullanılır. Dönüşüm, görüntünün farklı öneme sahip parçalara ayrılmasına yardım eder. İki görüntü, hash'leri arasındaki farklı bit sayısına bakılarak karşılaştırılır; bu Hamming uzaklığıdır. Hamming uzaklığı aynı uzunluktaki iki dizgiden birini diğerine dönüştürmek için gereken yer değiştirme sayısını, yani iki dizginin ne kadar farklı olduğunu verir.

10.2 Dört Aşama

Slayttaki şema (ekrandan, 02:30:00): girdi resim → dönüşüm → özellik çıkarma → (özellik vektörü) → niceleme → (ara nicelik vektörü) → sıkıştırma ve şifreleme → hash değeri.

1. **Dönüşüm:** $M \times N$ boyutundaki görüntüye renk dönüşümü, yumuşatma ya da DCT veya DWT gibi uzamsal dönüşümler uygulanır. Amaç, sonraki aşamada çıkarılacak öz nitelikleri piksel değerlerine ya da frekans katsayılarına bağlı hale getirmektir.
2. **Özellik çıkarma:** Dönüştürülmüş görüntüden bir özellik vektörü oluşturulur. Önce bütün öz nitelikler çıkarılır, sonra ilgili olanlar seçilip tutulur.
3. **Niceleme:** Sürekli algısal hash vektörünün her bileşeni (örneğin tek tip niceleme ile) nicelenerek ara bir vektör elde edilir.
4. **Sıkıştırma ve şifreleme:** İkili ara dizgi sıkıştırılır ve sabit boyutlu kısa bir algısal hash'e dönüştürülür.

10.3 pHash Oluşturma Adımları

1. **Renklerin azaltılması:** Hesabı basitleştirmek için görüntü gri tonlamaya çevrilir.
2. **Boyutun azaltılması:** Görüntü 32×32'ye küçültülür.
3. **DCT hesaplanması:** 32×32'lik DCT ile görüntü frekans ve skaler bileşenlerine ayrılır. DCT önce her satıra, sonra her sütuna uygulanır; en düşük frekanslar sol üst köşede toplanır.
4. **Düşük frekansların seçilmesi:** Sol üst köşedeki 8×8'lik kısım tutulur.
5. **Ortalamanın hesaplanması:** Bu katsayıların ortalaması alınır.
6. **Hash'in oluşturulması:** Ortalamadan büyük olanlara 1, küçüklere 0 yazılarak hash elde edilir.

10.4 Uygulama Alanları ve Sınırlılıklar

Sunumda sayılan kullanım alanları:

- **Telif hakkı ihlali:** Kırpma ya da ölçek küçültme ile değiştirilmiş olsalar bile benzer görüntü, ses ya da videolar eşleştirilebilir.
- **Yazım hatası düzeltme:** Metinsel algısal hash, sözcükleri seslerine göre kategorize ederek yanlış yazılmış sözcükleri yakalayıp düzeltmeyi mümkün kılar.
- **Giysi ve ekipman tespiti:** İnsanların ne giydiğini algılayan algoritmalar şantiye ve hastanelerde koruyucu ekipman giymemiş kişileri belirlemeye yardım edebilir.
- **Hastalık belirtisi tespiti:** Zamanında tespit edilmezse ciddi kayıplara yol açabilen lekelerin tespitinde algısal hash tabanlı görüntü işleme kullanılır.
- **Ters görsel arama:** E-ticarette bir fotoğrafla tam eşleşen ya da en benzer ürünü bulmak.
- **DNA dizileme:** Her nükleotit, önemli frekans özelliklerinden hesaplanan sabit gri seviyeli bir piksel olarak kodlanır; dizi ile algısal hash arasında ciddi veri azalması olur ve milyonlarca kombinasyon düşük altyapı maliyetiyle karşılaştırılabilir.
- **Akademik yazılarda görüntü intihali:** Boyutu küçültülmüş ya da döndürülmüş görüntüler tespit edilebilir.

Algısal hash fonksiyonları genellikle doğru çalışsa da yanlış eşleşme üretme eğilimindedir. Sunumda yüz tanıma yazılımlarının bazen ebeveyni çocuğuyla karıştırabildiği örneği verildi.

Son örnekte bir fotoğrafın üzerine yazı eklenmişti. İlk ve ikinci resmin hash'leri karşılaştırıldığında üç bitin değiştiği görüldü; Hamming uzaklığı 3'tü. Yani yazı eklenmesine rağmen hash büyük ölçüde korunmuştu.

11. Hash Algoritmalarının Güçlü ve Zayıf Yönleri (On Birinci Sunum)

Bu sunum, hash fonksiyonlarını saldırılara karşı karşılaştıran bir makaleyi ve 2022'de yayımlanmış ikinci bir makaleyi özetledi. İki kişilik grubun sunumunu bir kişi yaptı.

11.1 Motivasyon

Dijital ve fiziksel dünya arasındaki çizgi hızla değişirken kuruluşların hassas verilerini siber saldırılara karşı koruması her zamankinden önemlidir. Sunumda 2020'de kuruluşların %86'sının siber saldırıya maruz kaldığı ve bunların %69'unun fidye yazılımıyla ele geçirildiği istatistiği verildi. Soru şuydu: hangi hash fonksiyonları daha güvenilir, daha verimli? Makale hash fonksiyonlarını, türlerini, güçlü ve zayıf yönlerini karşılaştırıyor; zayıf yönlere karşı başka makalelerden yöntemler öneriyordu. Hash fonksiyonu, bir girdiyi okunamaz bir veri dizisine dönüştüren tek yönlü matematiksel bir fonksiyon olarak tanımlandı; kavramın 1953'te ortaya atıldığı söylendi.

11.2 Algoritmaların Avantaj ve Dezavantajları

Mesaj özetleme algoritması (MD5). Orijinal girdi sabit boyutlu bloklara bölünür; her blok bir önceki turun çıktısıyla birlikte sıkıştırma fonksiyonunda işlenir; sonuç 128 bitlik bir özettir. İlk yıllarda güvenli kabul edildi, sonra zayıf yönleri yüzünden kabul görmedi. Avantaj: küçük hash'leri karşılaştırmanın ve saklamanın kolay yolu; dosya ve kod karşılaştırmada kullanışlı. Dezavantaj: daha az güvenli ve saldırılara açık.

SHA-1. Herhangi uzunluktaki mesajı 160 bitlik, 40 haneli onaltılık bir değere dönüştüren tek yönlü bir fonksiyondur; yapısı MD5'e benzer. Dijital imza ve sertifikalar için kullanılmamalıdır: 2005'ten beri teorik olarak kırık sayılır, 2011'de resmi olarak kullanımdan kaldırılmış, 2017'de aynı SHA-1 değerine sahip iki dosya üreten Google araştırmacılarınca pratikte kırılmıştır. Yalnız kasıtsız bozulmaları belirlemek için dosya ya da kod karşılaştırmada, eski kodlarla uyumluluk gerektiğinde kullanılabilir. Dezavantajları: çıktı uzunluğu saldırılara direnmek için kısa, MD5'ten de yavaş, birçok güvenlik açığı bulunmuş.

SHA-2. SHA-1'e ek olarak önerilmiştir; farklı özet boyutlarına sahip altı algoritmadan oluşan bir ailedir: SHA-224, SHA-256, SHA-384, SHA-512 ve SHA-512'nin kısaltılmış iki sürümü (SHA-512/224 ve SHA-512/256). SHA-256 ABD devlet kurumlarınca hassas verileri korumak için yaygın kullanılır; makalenin yazıldığı tarihte mevcut en güvenli hash algoritması olarak belirtilmiştir ve Bitcoin işlemlerinde de yaygındır. Avantajları: SHA-1'in zayıf yönlerinin güçlendirilmiş hâlidir; tarayıcılar, işletim sistemleri, e-posta istemcileri ve mobil cihazlarca desteklenir; daha uzun değer ürettiği için sertifika ve dosya imzalama ve doğrulamada en etkili algoritma sayılabilir. Dezavantajı: öncekilerden yavaştır ve bazı sistemlerin SHA-2 desteği için güncellenmesi gerekebilir.

SHA-3. SHA ailesinin en son üyesidir; SHA-1 ve SHA-2'den tamamen farklı olmakla birlikte aynı standardın parçası olarak geliştirilmiştir. Sünger yapısını kullanır: veri sünger tarafından emilir, ardından tıpkı suyunu bırakan bir sünger gibi sıkılarak sonuç alınır. Dört sabit uzunluklu algoritma ve iki genişletilebilir çıktı fonksiyonundan (XOF) oluşan bir ailedir; XOF'lar alan hash'i, rastgele hash, akış şifreleme ve mesaj doğrulama kodu üretiminde kullanılabilir. Avantajları: kriptografik donanımla işlendiğinde öncekilerden hızlıdır; geniş kapsamlı saldırılara karşı en güçlü algoritmadır; uygun kapasite-oran (capacity-rate) çifti seçilerek performans ve güvenlik dengelenebilir; bütün güvenlik seviyeleri için tek bir permütasyon kullandığından uygulama maliyeti düşüktür; değişken uzunlukta girdi ve çıktıya izin verdiği için esnektir. Dezavantajları: sunumda çakışma saldırılarına karşı hassas olduğu, yazılımda SHA-2'den yavaş olduğu ve donanım ve yazılım desteğinin eksik olduğu söylendi.

> **Ders sonrası not:** Altyazıda SHA-3'ün XOF'ları için "MAC adresleri oluşturmak" ifadesi geçiyor; burada kastedilen ağ kartı adresi değil, mesaj doğrulama kodudur (Message Authentication Code, MAC).

Slayttaki karşılaştırma tablosu (ekrandan, 02:40:00):

	MD5	SHA-1	SHA-2	SHA-3
Yayın yılı	1992	1995	2002	2008
Blok boyutu	512 bit	512 bit	512/1024 bit	1152/1088/832/576 bit (oran, rate)
Özet boyutu	128 bit (32 onaltılık hane)	160 bit (40 hane)	256 bit (64 hane) / 512 bit (128 hane)	224/256/384/512 bit
Tur sayısı	64	80 (20'şerlik 4 grup)	64 (SHA-224, SHA-256) / 80 (SHA-384, SHA-512)	24
Yapı	Merkle-Damgård	Merkle-Damgård	Merkle-Damgård	Sünger (Keccak)
Çakışma düzeyi	Yüksek; sıradan bir ev bilgisayarıyla saniyeler içinde bulunabilir	Ucuz ve kolay bulunur	Düşük; bilinen çakışma yok	Düşük
Başarılı saldırılar	Çok; 2004'te somut kanıt	Çok; ilki 2017'de SHAttered	SHA-256 hiç kırılmadı	Az sayıda çakışma türü saldırı
Ortak zayıflık	Çakışmaya açık	Çakışmaya açık	Ön görüntü saldırılarına duyarlı	Pratik çakışma ve yakın çakışma saldırıları
Güvenlik düzeyi	Düşük	Düşük	Yüksek	Yüksek

> **Ders sonrası not:** SHA-3 standardı (FIPS 202) büyük olasılıkla 2015'te yayımlanmıştır; tablodaki 2008, Keccak'ın yarışmaya sunulduğu yıla karşılık gelir.

11.3 Gelecek ve Kuantum Hesaplama

Makale algoritmaların geleceğini de tartışıyordu. Dijital para birimlerinden artırılmış ve sanal gerçekliğe, nesnelerin internetinden (IoT) metaverse'e ve kuantum hesaplamaya kadar yeni ekosistemler son derece güvenli algoritmalara ihtiyaç duyuyor. Asıl soru hash algoritmalarına ihtiyaç olup olmadığı değil, SHA-256 gibi bugün güvenli sayılan algoritmaların gelecekteki zorluklara, özellikle kuantum hesaplamaya dayanıp dayanamayacağıdır. Makaleye göre buna şimdilik kesin bir cevap verilemiyor.

11.4 İkinci Makale: Önerilen Yöntemler

2022'de yayımlanan ikinci makale aynı algoritmaları anlatıp ek olarak üç yöntem önermiş, ama bunları ayrıntılı incelememişti. Makalede hash algoritmalarının çakışma oranları verilmiş ve literatürdeki yeni yöntemler karşılaştırılmıştı.

Karşılaştırma tablosu (ekrandan, 02:50:00):

Çalışma	Yıl	Avantaj	Sınırlılık
Belfedhal ve Faraoun	2015	Sözde rastgele davranış ve girdi değişikliklerine duyarlılık gibi iyi kriptografik özellikler	Ortadaki adam (meet-in-the-middle), çakışma ya da doğum günü gibi kriptografik saldırılara karşı test edilmedi
Wang ve ark.	2015	Değişken çıktı	Çakışma gibi yaygın saldırılara karşı test edilmedi

Çalışma	Yıl	Avantaj	Sınırlılık
Li ve ark.	2016	İyi istatistiksel performans ve çakışma direnci	Çıktı 128 bit olduğu için kapsamlı çakışma saldırısına açık
Tur ve Javurek	2016	(belirtilmemiş)	Gerçek uygulama için ek modüller gerekli; kriptografik saldırılara karşı test edilmedi
Li ve Liu	2016	Karıştırma (confusion) ve yayılma (diffusion) özelliği iyi	Çıktı 128 bit olduğu için kapsamlı çakışma saldırısına açık
Ahmad ve ark.	2017	Çok iyi istatistiksel performans; 160, 256 ya da 512 bit hash üretebilir	(belirtilmemiş)
Zhang ve ark.	2017	İstatistiksel performans gereksinimini karşılıyor	Hash değerini elde etmek zaman açısından verimli değil

Sunumda bu çalışmalar şöyle özetlendi: Belfedhal ve Faraoun 256 bitlik hash üreten, Merkle-Damgård yapısının bir varyantını kullanan bir fonksiyon önermiş. Li ve arkadaşları 128 bit hash üreten karmaşık bir fonksiyon tasarlamak için dinamik S-kutusu (S-box) yöntemini önermiş; S-kutusunun getirdiği esneklik en büyük dezavantaj olarak belirtilmiş ve hash uzunluğunun çakışma ya da ön görüntü saldırılarına karşı güvenliği garanti etmediği vurgulanmış. 224 bitlik özet üreten başka bir çalışmada özet üretmenin ciddi zaman alması verimliliği düşürüyor. Tur ve Javurek 128 bitlik hash için sinir ağı kullanmış; kısa hash değeri bir avantaj, ama yürütmede zorluklar var. Son çalışma 2B ve 1B kaotik haritaları birleştirerek rastgele uzunluktaki mesajdan 128 bit hash üretiyor; hash kısa olduğu için çakışmalara dirençli olmadığı savunulmuş.

12. SHA-256'nın Adımları ve Blokzincir (On İkinci Sunum)

12.1 Hash Fonksiyonunun Özellikleri ve Örnekler

Son sunum da blokzincirde hash fonksiyonları üzerineydi. Hash fonksiyonu girdiyi geri dönüşü olmayan (irreversible) biçimde dönüştürür; girdideki en küçük değişiklikte özet (digest) değeri anında değişir. Slaytlarda örnekler gösterildi: "Blockchain" ile "Blockchain is the Future" tamamen farklı çıktılar verdi; "hello" kelimesinde tek bir harf büyük yazılınca hash tamamen değişti. Girdiler sayı, metin, resim, video ya da ses olabilir; hepsi mesaja dönüştürülüp kriptografik hash fonksiyonundan geçirilir.

Sayılan özellikler:

- **Deterministik:** Aynı girdi her zaman aynı hash'i üretir; tek harf değişse bile hash önemli ölçüde değişir.
- **Neredeyse çakışmasız (nearly collision free).**
- **Tek yönlü:** x için $H(x)$ hesaplanabilir, ama $H(x)$'ten x bulunamaz.
- **Saldırı maliyeti:** Doğum günü saldırısı için yaklaşık $2^{n/2}$ hash işlemi gerekir. 256 bitlik bir hash için bu 2^{128} işlemdir; her işlem bir mikrosaniye sürse bile yaklaşık 10^{25} yıl eder.
- **Bilgi gizleme (information hiding):** Mesajın özeti geri döndürülemediği için bir mesajı doğrulamanın tek yolu aday mesajı özetleyip orijinal özetle eşleşip eşleşmediğine bakmaktır. Bu, bir mesajın özetini saklayıp daha sonra kontrol etmeye (taahhüt) yarar.

12.2 SHA-256'nın Adımları

SHA-256 bir mesajın 256 bitlik özetini üreten güvenli bir hash algoritmasıdır. Adımlar:

1. **Dolgu bitlerinin eklenmesi:** Mesaj, toplam uzunluğu 512'nin bir katından tam 64 bit eksik olacak şekilde ek bitlerle doldurulur. L mesajın uzunluğu, P dolgu bit sayısı olmak üzere $L + P + 64 = n \cdot 512$ olur.
2. **Uzunluk bitlerinin eklenmesi:** Mesajın uzunluğunu ifade eden 64 bit eklenir; böylece toplam uzunluk tam olarak 512'nin katı olur (ekrandan, 03:00:00).
3. **Zincirleme değişkenlerinin başlatılması:** Mesaj n adet 512 bitlik bloğa bölünür: $M_1, M_2, \ldots, M_n$. İşlemler için ara bellek değerleri gerekir; 0-63 indeksli 64 onaltılık sabit "anahtar" (key) diye adlandırılır ve k ile gösterilir. Slayttaki listenin başı `k[0..63] := 0x428a2f98, 0x71374491, 0xb5c0fbcf, 0xe9b5dba5, ...` biçimindeydi (ekrandan, 03:00:00).
4. **Blokların işlenmesi:** Her 512 bitlik blok 16 adet 32 bitlik alt bloğa bölünür. 64 işlem turu vardır ve her turun çıktısı bir sonraki turun girdisi olur. k değerleri işlenen iterasyon sayısına bağlı olarak her blok için kullanılır. Döngü bütün n blok işlenene kadar tekrar eder; son sonuç SHA-256 özetidir.

   Ders sonrası not: Slaytta uzunluk alanı "$L \bmod 2^{32}$" olarak yazılmış (ekrandan, 03:00:00). Uzunluk alanı 64 bit olduğu için doğrusu büyük olasılıkla $L \bmod 2^{64}$'tür. Ayrıca slayttaki $L + P + [L \bmod 2^{32}] = n \cdot 512$ ifadesinde üçüncü terim uzunluk alanının değeri değil, bit sayısı olan 64 olmalıdır.

Sunumun sonunda blokzincirde hash'in kullanım alanları sayıldı: Merkle ağacı, Proof of Work uzlaşısı, dijital imzalar ve blokların zincirlenmesi.

Örnek Sorular

Soru 1 (derste sözlü, 00:06:39): $H(x) = x \bmod 3$ fonksiyonu için bir çakışma, 2 çıktısının bir ön görüntüsü ve 8 girdisinin bir ikinci ön görüntüsü bulun.

Cevap (derste verildi): $5 \bmod 3 = 2$ ve $8 \bmod 3 = 2$ olduğundan $(5, 8)$ bir çakışmadır. 2 çıktısını veren 5 bir ön görüntüdür. 8 verildiğinde $H(8) = 2$'yi veren başka bir girdi olarak 5 bir ikinci ön görüntüdür. Bu fonksiyon üç saldırıya karşı da dirençsizdir.

Soru 2 (ekrandan, 00:28:45): Saniyede yaklaşık 60 milyon hash hesaplayan normal bir bilgisayar, beklenen deneme sayısı 2^{255} olan bir arama için ne kadar süre harcar?

Cevap (derste verildi): Slayttaki sonuç: $2^{255}/(60 \cdot 10^6)$, yaklaşık $3 \cdot 10^{61}$ yıl.

Çözüm (derste verilmedi): $2^{255} \approx 5{,}79 \cdot 10^{76}$. Bunu $6 \cdot 10^7$'ye bölünce yaklaşık $9{,}65 \cdot 10^{68}$ saniye çıkar. Bir yıl yaklaşık $3{,}156 \cdot 10^7$ saniye olduğundan süre yaklaşık $3{,}06 \cdot 10^{61}$ yıldır (python ile doğrulandı). Altyazıda geçen "2 üzeri 55" ile hesap yapılsaydı sonuç yalnız yaklaşık 19 yıl olurdu; slayttaki doğru üs 255'tir.

Soru 3 (derste sözlü, 00:32:50): Algısal hash'te dayanıklılık (resilient) özelliği hangi uygulamalar için önemlidir?

Cevap (derste verildi): Nesne takibi için önemlidir. Nesne hareket ederken yalnız konumu değişir; ardışık karelerdeki hash'lerin birbirine yakın olması gerekir. Bulanıklaştırılmış ya da biraz değiştirilmiş bir görüntünün içerik olarak aynı tanınması da bu özelliğe dayanır.

Soru 4 (ekrandan, 01:10:00): H evrensel bir hash ailesi ve n anahtar m slotlu bir tabloya hash'lenmiş olsun. Bir x anahtarıyla çakışan anahtarların beklenen sayısı en fazla nedir?

Cevap (derste verildi): $C_x = \sum_{y \neq x} c_{xy}$ ve beklenen değerin doğrusallığından $E[C_x] = \sum_{y \neq x} E[c_{xy}] \leq \sum_{y \neq x} \frac{1}{m} = \frac{n-1}{m}$. Bu değer n/m'den küçüktür.

Soru 5 (ekrandan, 01:11:50): Asal sayı yöntemindeki $h_{a,b}(x) = ((ax + b) \bmod p) \bmod m$ ailesinde kaç fonksiyon vardır? Neden $a = 0$ seçilmez?

Cevap (derste verildi): $a \in \{1, \ldots, p-1\}$ için $p-1$, $b \in \{0, \ldots, p-1\}$ için p seçenek vardır; $|H| = p(p-1)$. $a = 0$ olursa her anahtar için sonuç b mod m olur, yani bütün anahtarlar aynı slota düşer ve her çift çakışır.

Soru 6 (derste sözlü, 01:36:07): 23 kişilik bir grupta iki kişinin doğum gününün aynı olma olasılığı nedir? 40-50 kişide ne olur?

Cevap (derste verildi): 23 kişide yaklaşık %50, 40-50 kişide %90'ın üzerinde.

Çözüm (derste verilmedi): Hiç çakışma olmama olasılığı $\prod_{i=0}^{k-1} \frac{365-i}{365}$'tir; aranan olasılık bunun 1'e tümleyenidir. $k = 23$ için 0,507; $k = 40$ için 0,891; $k = 41$ için 0,903; $k = 50$ için 0,970 çıkar (python ile hesaplandı). Yani %90 eşiği 41 kişide aşılır. Hash'te aynı mantıkla n bitlik çıktı için yaklaşık $2^{n/2}$ denemede çakışma beklenir.

Soru 7 (derste sözlü, 02:57:06): 256 bitlik bir hash için doğum günü saldırısı kaç işlem gerektirir? Her işlem bir mikrosaniye sürerse ne kadar zaman alır?

Cevap (derste verildi): $2^{256/2} = 2^{128}$ işlem; yaklaşık 10^{25} yıl.

Çözüm (derste verilmedi): $2^{128} \approx 3{,}40 \cdot 10^{38}$ işlem, 10^{-6} saniyeyle çarpılınca yaklaşık $3{,}40 \cdot 10^{32}$ saniye eder; bu da yaklaşık $1{,}08 \cdot 10^{25}$ yıldır (python ile doğrulandı).

Dersin Toparlaması

Bu derste on iki proje sunumu dinlendi ve hepsi hash fonksiyonları etrafında toplandı. Kriptografik tarafta bir hash fonksiyonundan beklenen üç direnç (çakışma, ön görüntü, ikinci ön görüntü), doğum günü sınırı $2^{n/2}$, Merkle-Damgård ve sünger yapıları ile MD5, RIPEMD-160, SHA-1, SHA-2 ve SHA-3'ün yapıları, tur sayıları, bilinen saldırıları ve teorik güvenlik bitleri karşılaştırıldı. MD5 ve SHA-1'in pratikte kırıldığı, SHA-2 ve SHA-3'ün bugün güvenli sayıldığı, yeni önerilen yöntemlerin ise çoğu zaman yeterince test edilmediği ya da 128 bit gibi kısa çıktı nedeniyle zayıf kaldığı vurgulandı.

Algoritma analizi açısından en kuramsal sunum evrensel hash üzerineydi: rastgele seçilen bir fonksiyon için $\Pr[h(x) = h(y)] \leq 1/m$ tanımı, beklenen çakışma sayısının $(n-1)/m$ olduğunun ispatı ve asal sayı, matris ve polinom yöntemleriyle evrensel aileler kuruldu. Algısal hash sunumları ise kriptografik hash'in tersine, benzer içeriğe benzer hash veren aHash, dHash, pHash, wHash, LHash ve LDHash yöntemlerini ve bunların içerik moderasyonu, nesne takibi ve telif tespiti gibi uygulamalarını anlattı. Blokzincir sunumları hash'in önceki bloğa bağlama, Proof of Work, dijital imza ve SHA-256'nın adımlarıyla nasıl kullanıldığını gösterdi. Hoca sunumlardan genel olarak güncel çalışmalara değinilmesini ve literatürün kaynak adlarıyla birlikte verilmesini bekledi.

Not: İsterseniz bu kayıtları Gemini Notebook gibi bir araca atıp daha istifadeli hale getirebilirsiniz.